

Day 1: 간이 로그인 검증 프로그램 & Gradle 빌드

목표: Gradle 프로젝트 구조를 이해하며, 평문 비밀번호 저장의 한계를 체감합니다.

1. 프로젝트 생성

1. VS Code에서 **Cmd + Shift + P**를 눌러 명령어 팔레트를 엽니다.
2. **Java: Create Java Project**를 검색하여 선택합니다.
3. 빌드 도구로 **Gradle**을 선택합니다.
4. DSL 선택 창에서 **Groovy**를 선택합니다.
5. 현재 열려있는 작업 폴더를 선택합니다.
6. 프로젝트 이름에 **auth-server**를 입력하고 엔터를 누릅니다.
7. 생성이 완료되면 오른쪽 하단의 "Open" 버튼을 누르거나 생성된 폴더로 이동합니다.
8. 좌측 탐색기에서 **app/src/main/java/** 경로를 찾습니다.
9. 해당 폴더 안에 있는 기본 파일(**App.java** 등)이 있다면 삭제합니다.
10. **java** 폴더를 우클릭하고 **New Java Package...**를 선택한 후, **com.example.auth**를 입력합니다.
11. 방금 만든 **com.example.auth** 패키지 폴더를 우클릭하고 **New Java File -> Class**를 선택한 뒤 **App**이라고 입력합니다.

2. 소스 코드 작성

app/src/main/java/com/example/auth/App.java 파일을 열고 다음 코드로 작성합니다.

```
package com.example.auth;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class App {
    // 메모리 DB 역할을 하는 Map (평문 저장의 문제점 체감용)
    private static final Map<String, String> db = new HashMap<>();

    static {
        db.put("alice", "pass1234");
        db.put("bob", "mySecret");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Day 1: 간이 로그인 시스템 ===");

        System.out.print("아이디: ");
        String id = sc.nextLine().trim();
```

```
System.out.print("비밀번호: ");
String pw = sc.nextLine().trim();

if (!db.containsKey(id)) {
    System.out.println("존재하지 않는 ID입니다.");
    return;
}

// 평문 비교 (보안 취약점)
if (db.get(id).equals(pw)) {
    System.out.println("로그인 성공! 환영합니다.");
} else {
    System.out.println("비밀번호가 틀렸습니다.");
}
}
```

3. 빌드 및 실행 (터미널)

```
cd app

# 빌드 수행 (Executable JAR 생성)
./gradlew clean build

# 생성된 JAR 파일 직접 실행
java -jar build/libs/app.jar
```